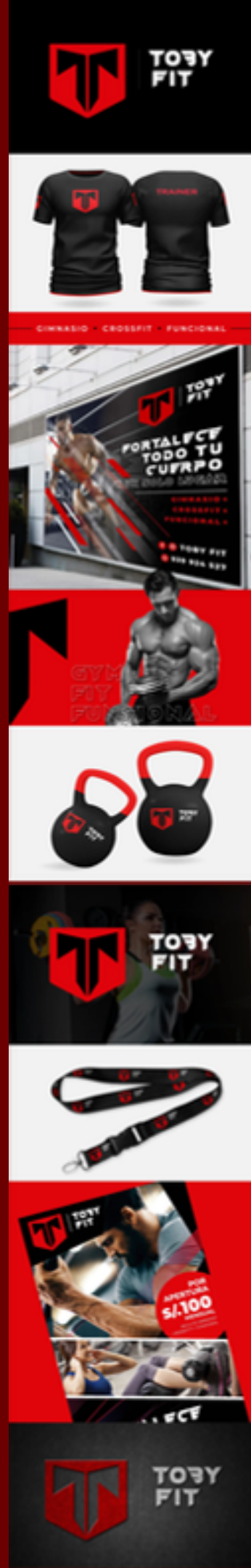


CURSO: DESARROLLO WEB INTEGRADO
ING. PAUL MAURICIO MENDOZA DEL CARPIO

PRESENTACIÓN DEL PRIMER AVANCE

INTEGRANTES

1. BENITES DÁVILA, Nayeli - U22209519
2. CARBONELL COLLAZOS, Fabrizzio Eduardo - U22211794
3. CORREA ANGELES, Daleth - U23225485
4. ESPINOZA BERRIOS, Julio Cesar - U23306078
5. TIZNADO VELÁSQUEZ, Héctor Fabrizzio - U23258394







GYM TOBY FIT

Plataforma Web Integral
Gestión de Gimnasio, Tienda y Membresías
en un solo lugar.



BIENVENIDOS

¡TU GIMNASIO 24/7 EN VENTANILLA!

Ex zona comercial

3er piso del Footloose (Ref. Paradero Teléfono)





1. PRESENTACIÓN Y CONTEXTO

Propósito del Proyecto: Reemplazar la gestión manual mediante una plataforma web centralizada orientada a servicios REST para administración de catálogo y clientes.

Despliegue Backend: Implementado sobre Spring Boot 3.4 con arquitectura en capas (Controller, Service, Repository, Entity).

Ventaja Competitiva: Soporte de stock en tiempo real, integración fluida con clientes SPA/Mobile y contratos de datos seguros vía JSON.





2. ALCANCES Y OBJETIVOS

OBJETIVO GENERAL

Implementar una API REST robusta en Java con Spring Boot para la gestión de productos y usuarios del gimnasio TOBY FIT, garantizando mantenibilidad y alta cohesión.

SERVICIOS & CONTROLLERS

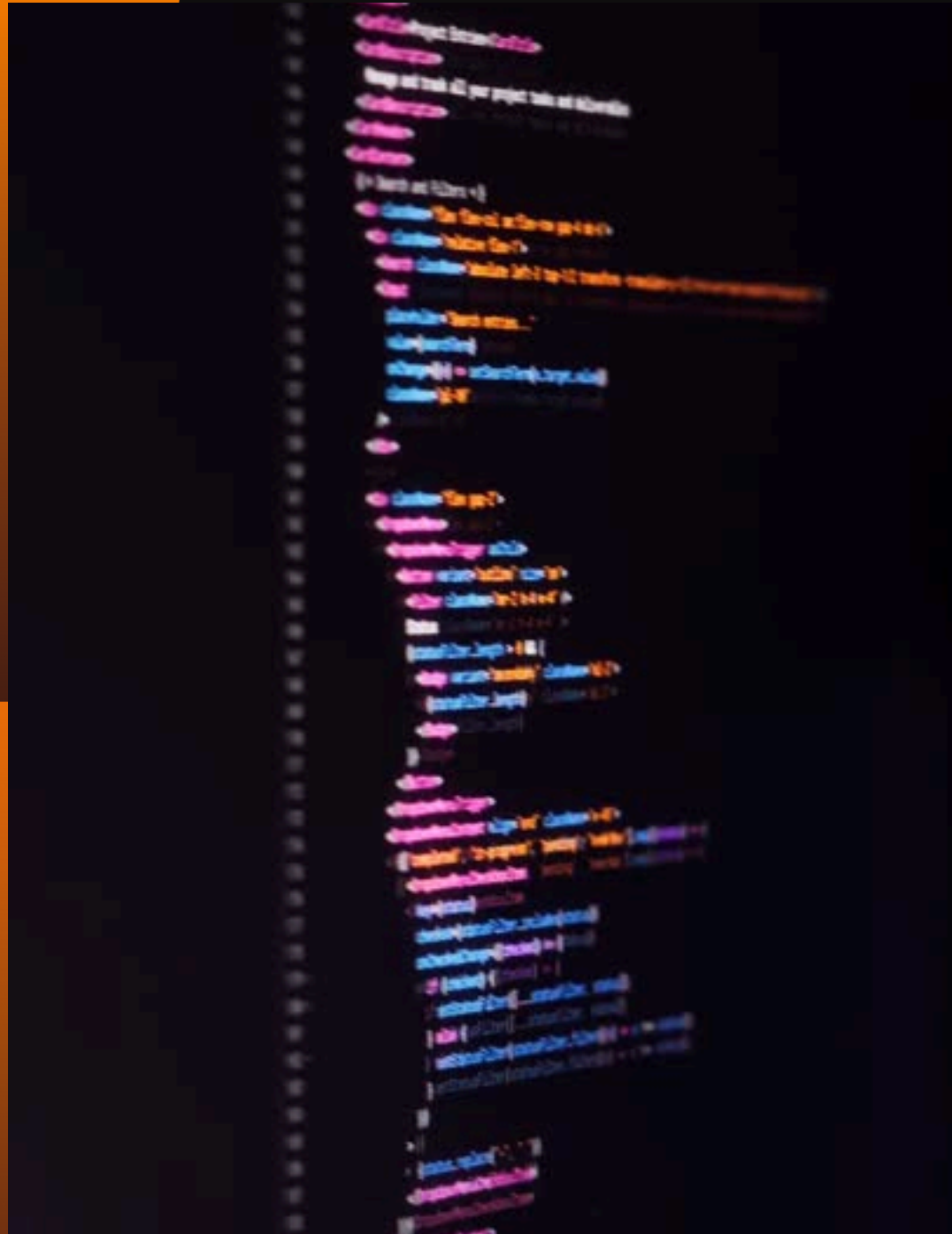
Construir controladores RESTful (ProductoApiController y UsuarioApiController) que manejen peticiones HTTP GET, POST, PUT y DELETE con respuestas parametrizadas.

ASEGURAMIENTO MOCKMVC

Verificar el 100% de los endpoints mediante 24 pruebas automatizadas con JUnit 5 y Surefire, simulando payloads JSON y validando respuestas de estado HTTP.



PREPARAR EL ENTORNO LOCAL



01

JDK 21 y Java Home

Instalar JDK 21 y definir la variable JAVA_HOME en el sistema antes de ejecutar el proyecto.

02

Base de Datos y Editor

MySQL · XAMPP con phpMyAdmin como opción local · VS Code, Eclipse

03

Maven Wrapper Incluido

El proyecto incluye mvnw.cmd: no se requiere instalación manual de Maven. Ejecutar desde la carpeta.

3. ENTIDADES DEL SISTEMA

(MODELO DE DATOS)

Modelo Java	N° Atrib.	Atributos Principales y Tipos	Soporte en API REST
Producto	9	id (Long), nombre (String), precio (BigDecimal), categoria (String)	Fase 3 Activo
Usuario	8	id (Long), correo (String), rol (Enum), telefono (String)	Fase 3 Activo



QUÉ EXIGE LA EVALUACIÓN TÉCNICA

8

Operaciones REST

Se requieren al menos 8 endpoints REST funcionales con métodos y rutas definidos en controladores.

2

Controladores como mínimo

4 objetos DTO con 4 o más atributos · 24 pruebas: 3 por operación.

24

Pruebas: 3 por operación

Nota: La descripción recibida no incluye los controladores ni los reportes de pruebas.

4. CONTROLLERS + CODIGO SPRING BOOT (OPERACIONES REST)

UsuarioApiController.java

Es un controlador REST encargado de gestionar las operaciones relacionadas con los usuarios del sistema. Utiliza UsuarioService para aplicar la lógica de negocio y expone tres servicios mediante API:

- GET /api/usuarios: obtiene la lista de usuarios registrados.
- PUT /api/usuarios/{id}: actualiza la información de un usuario existente validando que el ID coincida.
- DELETE /api/usuarios/{id}: elimina un usuario por su identificador.

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

```
ProductoApiController.java

@RestController
@RequestMapping("/api/productos")
public class ProductoApiController {

    private final ProductoService service;

    public ProductoApiController(ProductoService s) {
        this.service = s;
    }

    @GetMapping
    public List<Producto> listar() {
        return service.obtenerTodos();
    }

    @PostMapping
    public ResponseEntity<Producto> crear(@RequestBody Producto p) {
        return ResponseEntity.ok(service.guardar(p));
    }
}
```



```
UserController.java

@RestController
@RequestMapping({"/api/usuarios"})
public class UsuarioApiController {

    private final UsuarioService usuarioService;

    public UsuarioApiController(UsuarioService usuarioService) {
        this.usuarioService = usuarioService;
    }

    @GetMapping
    public ResponseEntity<List<Usuario>> listarUsuarios() {
        return ResponseEntity.ok(this.usuarioService.findAll());
    }

    @PutMapping("/{id}")
    public ResponseEntity<Usuario> actualizarUsuario(@PathVariable Long id, @RequestBody Usuario usuario) {
        if (usuario.getId() == null) {
            usuario.setId(id);
        }
        return ResponseEntity.ok(usuarioService.guardar(usuario));
    }
}
```

5. TECNOLOGÍAS

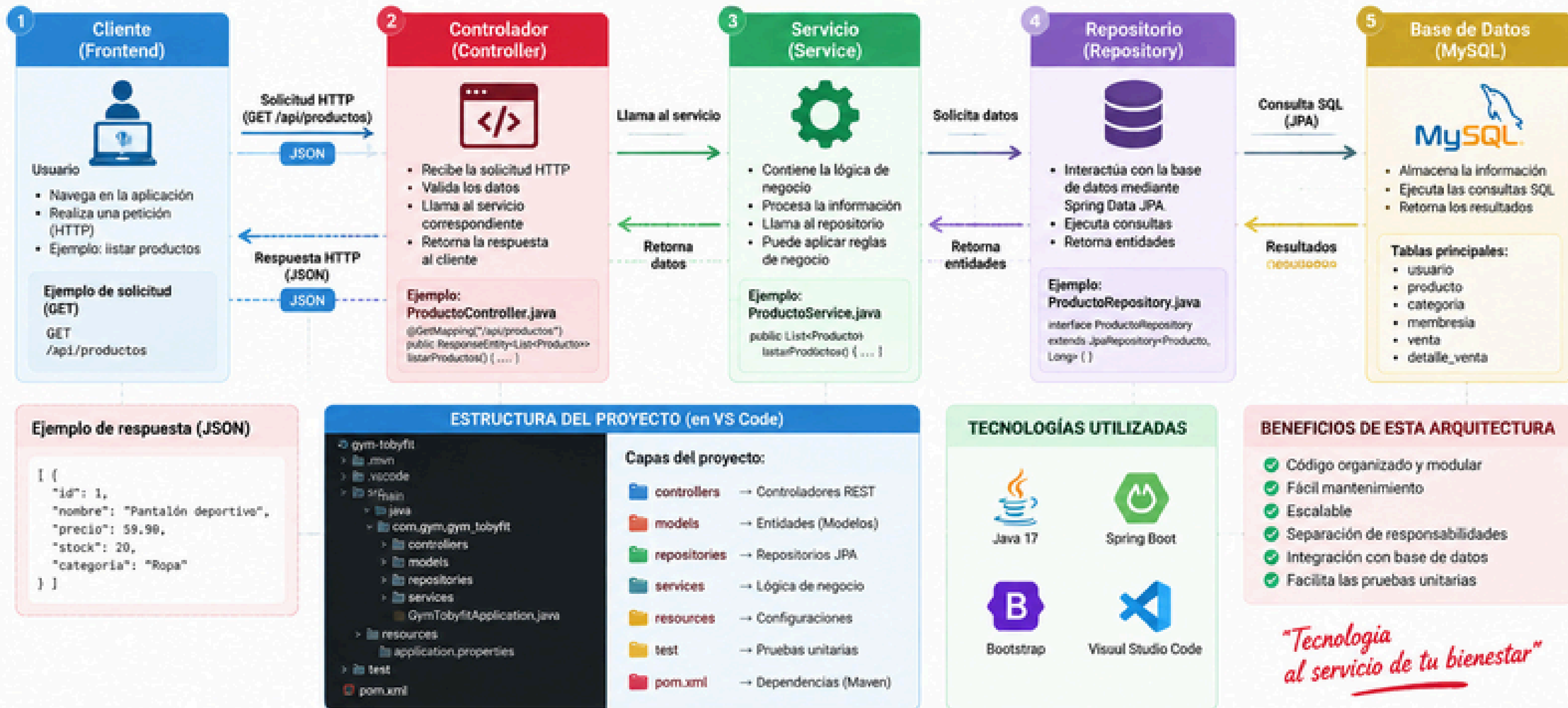
Elemento	F1 · Toby Fit	F2 · Rúbrica
Identificador Maven	gym-tobifit	gym-olympus
Spring Boot	3.4.10	3.4.10
Java	21	24
Lombok	1.18.48	1.18.48
Maven Wrapper	Maven 3.9.11	Maven 3.9.11
Base predeterminada	gym_tobyfit	gym_olympus
Clase principal	GymTobifitApplication	GymOlympusApplicati on
Dependencias principales	Web, Data JPA, MySQL Connector/J, Lombok y Starter Test.	Las mismas familias declaradas en F1.

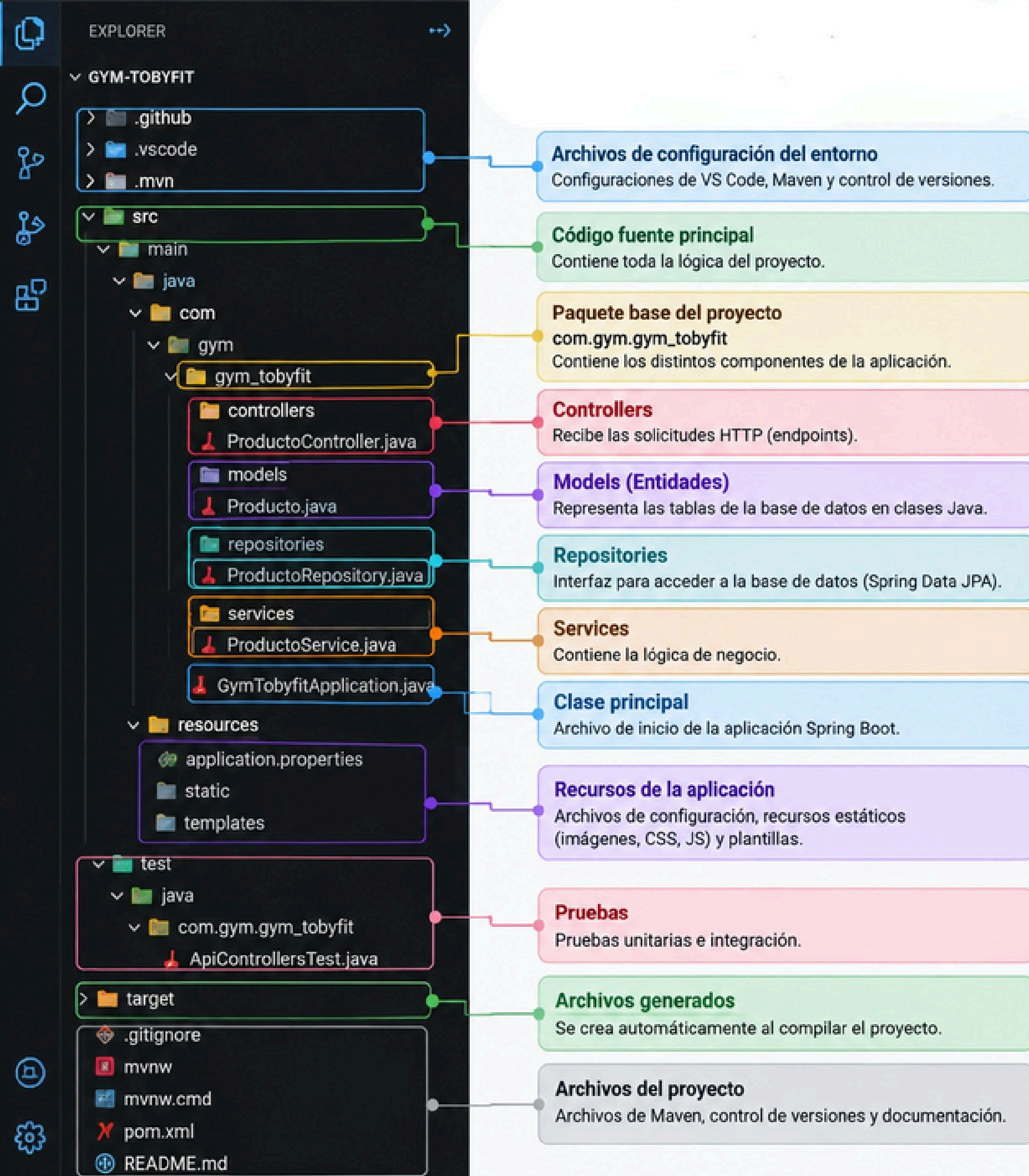
```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://w
3  <modelVersion>4.0.0</modelVersion>
4  <parent><groupId>org.springframework.boot</groupId><artifactId>spring
5  <groupId>com.gym</groupId><artifactId>gym-tobifit</artifactId><version>
6  <properties><java.version>21</java.version><lombok.version>1.18.48</l
7  <dependencies>
8    <dependency><groupId>org.springframework.boot</groupId><artifactId>sp
9    <dependency><groupId>org.springframework.boot</groupId><artifactId>sp
10   <dependency><groupId>com.mysql</groupId><artifactId>mysql-connector-j
11   <dependency><groupId>org.projectlombok</groupId><artifactId>lombok</l
12   <dependency><groupId>org.springframework.boot</groupId><artifactId>sp
13 </dependencies>
14 <build><plugins>
15   <plugin><groupId>org.apache.maven.plugins</groupId><artifactId>maven
16   <plugin><groupId>org.springframework.boot</groupId><artifactId>spring
17 </plugins></build>
18 </project>
```

pom.xml, application.properties y configuración del wrapper de cada ZIP. Las versiones declaradas no certifican la resolución de todas las dependencias en un entorno nuevo.

6. FLUJO DEL PROYECTO

"De una solicitud, a grandes resultados"





ARQUITECTURA DEL PROYECTO

La estructura está compuesta por:

- **Controllers**: Gestionan las solicitudes HTTP y exponen los servicios mediante API REST.
- **Services**: Contienen la lógica de negocio y procesan las operaciones del sistema.
- **Repositories**: Permiten la comunicación con la base de datos mediante Spring Data JPA.
- **Models**: Representan las entidades del sistema y la estructura de las tablas de la base de datos.
- **Resources**: Contienen archivos de configuración como **application.properties** y recursos del aplicativo.
- **Tests**: Incluyen pruebas para validar el correcto funcionamiento de los endpoints implementados.

7. CONFIGURACIÓN

MODELOS DEL DOMINIO JPA (MODELS)

Daleth Correa

```
Producto.java X
gym-tobyfit > src > main > java > com > gym > gym_tobifit > models > Producto.java > ...
1 package com.gym.gym_tobifit.models;
2 import jakarta.persistence.*;
3 import lombok.*;
4 import java.math.BigDecimal;
5 @Entity @Table(name="productos") @Data @NoArgsConstructor @AllArgsConstructor
6 public class Producto {
7     @Id @GeneratedValue(strategy=GenerationType.IDENTITY) private Long id;
8     @Column(nullable=false,length=100) private String nombre;
9     @Column(nullable=false,length=50) private String categoria;
10    @Column(nullable=false,length=20) private String genero;
11    @Column(nullable=false,precision=10,scale=2) private BigDecimal precio;
12    @Column(nullable=false,length=255) private String imagenPrincipal;
13    @Column(columnDefinition="TEXT") private String imagenes;
14    @Column(columnDefinition="TEXT") private String modelos;
15    @Column(columnDefinition="boolean default true") private Boolean activo=true;
16 }
```

- Entidad mapeada a MySQL mediante JPA/Hibernate. Cuenta con 9 atributos para catálogo comercial.
- Mapeo de Precio: Usa BigDecimal para evitar errores de redondeo monetario.
- Atributos Clave: id, nombre, categoria, genero, precio, imagenPrincipal, modelos, activo.

8. CAPA DE SERVICIOS Y PERSISTENCIA JPA

Spring Data JPA genera dinámicamente las consultas SQL sobre MySQL a través de JpaRepository.

Hector Tiznado

```
ProductoService.java X
gym-tobifit > src > main > java > com > gym > gym_tobifit > services > ProductoService.java > {} com.gym.gym_tobifit.services

1 package com.gym.gym_tobifit.services;
2
3 import com.gym.gym_tobifit.models.Producto;
4 import com.gym.gym_tobifit.repositories.ProductoRepository;
5
6 import org.springframework.stereotype.Service;
7 import java.util.List;
8 import java.util.Optional;
9
10 @Service
11 public class ProductoService {
12     private final ProductoRepository productoRepository;
13     public ProductoService(ProductoRepository productoRepository) { this.productoRepository = productoRepository; }
14     public List<Producto> listarProductos() { return productoRepository.findAll(); }
15     public Optional<Producto> obtenerProductoPorId(Long id) { return productoRepository.findById(id); }
16     public Producto guardarProducto(Producto producto) { return productoRepository.save(producto); }
17     public Producto actualizarProducto(Producto producto) {
18         if (producto.getId() == null) throw new RuntimeException("El ID es obligatorio");
19         Producto actual = productoRepository.findById(producto.getId()).orElseThrow(() -> new RuntimeException("Producto no encontrado"));
20         if (producto.getNombre() != null) actual.setNombre(producto.getNombre());
21         if (producto.getCategoria() != null) actual.setCategoria(producto.getCategoria());
22         if (producto.getGenero() != null) actual.setGenero(producto.getGenero());
23         if (producto.getPrecio() != null) actual.setPrecio(producto.getPrecio());
24         if (producto.getImagenPrincipal() != null) actual.setImagenPrincipal(producto.getImagenPrincipal());
25         if (producto.getImagenes() != null) actual.setImagenes(producto.getImagenes());
26         if (producto.getModelos() != null) actual.setModelos(producto.getModelos());
27         if (producto.getActivo() != null) actual.setActivo(producto.getActivo());
28         return productoRepository.save(actual);
29     }
30     public void eliminarProducto(Long id) {
31         if (!productoRepository.existsById(id)) throw new RuntimeException("Producto no encontrado");
32         productoRepository.deleteById(id);
33     }
34 }
```

```
@Repository
public interface ProductoRepository
    extends JpaRepository<Producto, Long> {

    List<Producto> findByActivoTrue();
}
```

XAMPP Control Panel v3.3.0 [Compiled: Apr 6th 2021]

XAMPP Control Panel v3.3.0

Service	Module	PID(s)	Port(s)	Actions
☐	Apache	18688 3156	80, 443	Stop Admin Config Logs
☐	MySQL	10000	3306	Stop Admin Config Logs
☐	FileZilla			Start Admin Config Logs
☐	Mercury			Start Admin Config Logs
☐	Tomcat	16492	60454, 62746, 62967,	Stop Admin Config Logs

09:16:00 [mysql] entire log window on the forums
 09:19:54 [mysql] Attempting to start MySQL app...
 09:19:54 [mysql] Status change detected: running
 09:21:08 [Apache] Attempting to start Apache app...
 09:21:09 [Apache] Status change detected: running
 09:25:01 [Tomcat] Status change detected: running
 09:47:29 [Tomcat] Status change detected: stopped
 09:49:54 [Tomcat] Status change detected: running

Config Netstat Shell Explorer Services Help Quit

BASE DE DATOS XAMPP SQL



phpMyAdmin

Importación ejecutada exitosamente, 19 consultas ejecutadas. (gym_tobyft.sql)

MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0.0003 segundos.)

-- phpMyAdmin SQL Dump -- version 5.2.1 -- https://www.phpmyadmin.net/ -- -- Servidor: localhost -- Tiempo de generación: 07-09-2026 a las 23:02:26 -- Versión del servidor: 10.4.32-MariaDB -- NO_AUTO_VALUE_ON_ZERO

[Editar en línea] [Editar] [Crear código PHP]

MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0.0003 segundos.)

START TRANSACTION;

[Editar en línea] [Editar] [Crear código PHP]

MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0.0002 segundos.)

SET time_zone = '+00:00';

[Editar en línea] [Editar] [Crear código PHP]

MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0.0002 segundos.)

/*148581 SET @OLD_CHARACTER_SET_CLIENT=@CHARACTER_SET_CLIENT */;

[Editar en línea] [Editar] [Crear código PHP]

MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0.0002 segundos.)

/*148581 SET @OLD_CHARACTER_SET_RESULTS=@CHARACTER_SET_RESULTS */;

[Editar en línea] [Editar] [Crear código PHP]

MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0.0002 segundos.)

/*148581 SET @OLD_COLLATION_CHARACTER_SET_NAME=@COLLATION_CHARACTER_SET_NAME */;



9.RESUMEN DE ENDPOINTS REST

Método	Ruta Endpoint	Controlador	Payload / Parámetros	Estado HTTP
GET	/api/productos	ProductoApiController	Ninguno → List	200 OK
GET	/api/productos/{id}	ProductoApiController	id: Long → Producto	200 OK / 404
POST	/api/productos	ProductoApiController	Producto (JSON Body)	200 OK
PUT	/api/productos/{id}	ProductoApiController	id + Producto (JSON Body)	200 OK / 404
DELETE	/api/productos/{id}	ProductoApiController	id: Long → Sin cuerpo	200 OK
GET	/api/usuarios	UsuarioApiController	Ninguno → List	200 OK
PUT	/api/usuarios/{id}	UsuarioApiController	id + Usuario (JSON Body)	200 / 400 / 404
DELETE	/api/usuarios/{id}	UsuarioApiController	id: Long → Sin cuerpo	200 OK / 404

10.PRUEBAS Y EJECUCIÓN POSTMAN (CONSULTAS)

24

Casos
Ejecutados

The image is a collage of Postman interface elements. A large blue 'X' is superimposed over the right side. A purple lightning bolt icon is in the bottom center. The background shows various Postman windows:

- Top Left:** A 'New Request' window for a GET request to `http://localhost:8080/api/product`. The status is 200, size is 4.09 KB, and time is 515 ms.
- Top Right:** A 'List de Usuarios' window showing a GET request to `http://localhost:8080/api/usuarios` with a 200 OK status.
- Bottom Left:** A 'Verify campos del usuario' window showing a GET request to `http://localhost:8080/api/usuarios` with a 200 OK status.
- Bottom Right:** A 'List de Usuarios' window showing a GET request to `http://localhost:8080/api/usuarios` with a 200 OK status.

The central part of the image displays a JSON response for a product:

```
1 [
2   {
3     "id": 1,
4     "nombre": "GYMRAT - Polo Oversize",
5     "categoria": "polo",
6     "genero": "hombre",
7     "precio": 49.00,
8     "imagenPrincipal": "/img/polo_hombre.png",
9     "imagenes": "[\"/img/polo_hombre.png\", \"/img/polo_hombre_1.png\"]",
10    "modelos": "[\"Negro\"]",
11    "activo": true
12  },
13  {
14    "id": 2,
15    "nombre": "GYMRAT - Polo Oversize",
16    "categoria": "polo",
17    "genero": "mujer",
18    "precio": 45.00
19  }
20 ]
```

REPORTE DE PRUEBAS SUREFIRE

OP-01: Listar Productos (CP 1-3)	3 / 3 Exitosos (100%)
OP-02: Obtener Producto (CP 4-6)	3 / 3 Exitosos (100%)
OP-03: Crear Producto (CP 7-9)	3 / 3 Exitosos (100%)
OP-04: Actualizar Producto (CP 10-12)	3 / 3 Exitosos (100%)
OP-05: Eliminar Producto (CP 13-15)	3 / 3 Exitosos (100%)
OP-06: Listar Usuarios (CP 16-18)	3 / 3 Exitosos (100%)
OP-07: Actualizar Usuario (CP 19-21)	3 / 3 Exitosos (100%)
OP-08: Eliminar Usuario (CP 22-24)	3 / 3 Exitosos (100%)



***A CONTINUACIÓN
VAMOS A POSTMAN***

CONCLUSIONES

- La migración a servicios REST desacopla el backend, facilitando futuras integraciones web y móviles.
- **Calidad Garantizada**, 100% de efectividad en las 24 pruebas automatizadas (JUnit 5 / MockMvc) reportadas por Surefire.
- La integración de Spring Data JPA simplificó la gestión del modelo de datos sobre MySQL.
- Validación total de la lógica de negocio y del manejo de respuestas HTTP (200 OK, 400 Bad Request, 404 Not Found).





2026

GRACIAS

POR

SU ATENCIÓN!